

Informe Tarea 1

Anne Murillo Mora, amurillo@usm.cl

Ivanna Chitty Gonzalez, Ichitty@usm.cl

Victoria Negrete Hermosilla, vnegrete@usm.cl

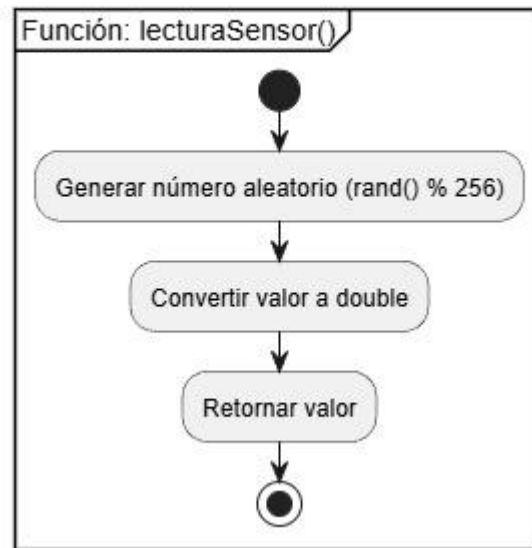
1. Parte 1

1.1. Diseño de la función `lecturaSensor()`

La función `lecturaSensor()` está diseñada para simular de manera autónoma la adquisición de datos de un sensor físico. Se caracteriza por no recibir parámetros, lo que garantiza su independencia y su alta eficiencia para evitar operaciones de bloqueo. Cada vez que se llama, retorna un valor de tipo *Double* que representa una magnitud en milivoltios (mV). Aunque el valor se genera mediante la operación `rand()%256` (la cual produce enteros de 0 a 255), luego se aplica una conversión explícita a *Double* para mantener la consistencia de los tipos de datos del sistema. Para que estos valores sean realmente variables y emulen un entorno real, es fundamental inicializar la “semilla” con `srand(time(null))` en el programa principal.

Por otro lado, la arquitectura del software utiliza el tipo de enumeración *enum Accion* como un pilar para la gestión de operaciones. Este enumerado define tres estados claros *INIT* (valor 0 para inicializar), *READ* (valor 1 para lectura) y *SHOW* (valor 2 para mostrar datos). El uso de *enum* en lugar de números enteros simples nos otorga 4 principales ventajas las cuales son, Claridad semántica (código legible y fácil de entender), Mantenibilidad (facilita la evolución del sistema, nos permite añadir funciones nuevas como *CALIBRATE* o *TEST* sin alterar el sistema original), Seguridad de tipos (El compilador valida que solo se utilicen funciones permitidas, para evitar errores por valores fuera de rango) y Auto documentación (el código explica su propio propósito, evitando la necesidad de documentación externa).

Diagramas de Flujo - Simulación de Sensores



El diagrama de flujo de la función `lecturaSensor()` presenta una estructura secuencial simple compuesta por tres pasos fundamentales, sin bifurcaciones ni bucles:

El sistema utiliza la función `rand()` de la librería estándar para generar un valor pseudoaleatorio. Al aplicar el operador módulo (`% 256`), el resultado se acota estrictamente al rango $[0, 255]$. Este intervalo no es arbitrario; emula la resolución de un convertidor analógico-digital (ADC) de 8 bits, donde cada unidad representa una medición en milivoltios (mV).

Aunque el valor obtenido es un entero, se somete a un moldeado explícito (casting) al tipo `Double`. Esta conversión garantiza la concordancia con la firma de la función. Además, el uso de coma flotante otorga al sistema la flexibilidad necesaria para futuras actualizaciones, como la integración de ruido fraccionario o el soporte para sensores de mayor precisión decimal que un simple entero no podría representar.

Finalmente, la magnitud procesada se envía al punto de llamada mediante la instrucción `return`. Este valor es capturado por la estructura de control de la función `usarSensores()` (específicamente en su estado `READ`), donde se almacena en el arreglo correspondiente para su posterior tratamiento o visualización.

Esta función está diseñada para trabajar específicamente dentro del bloque `READ` de `usarSensores`. Su operación principal consiste en recorrer un arreglo y almacenar los datos de `lecturaSensor()`. Aunque los valores se manejan temporalmente como `Double`, se les aplica un casteo explícito a `int` para guardarlos en el arreglo final. Esta conversión se justifica como una decisión de diseño para mantener la flexibilidad del código ante futuros cambios en el tipo de datos, priorizando la adaptabilidad de la interfaz por encima de la eficiencia inmediata de la memoria.

Diseño completo de la nueva función main()

La función main() actúa como el punto de entrada y el organizador del programa. Su diseño sigue un enfoque del tipo modular, lo que significa que en lugar de hacer todo el trabajo por su cuenta, delega tareas específicas a la función usarSensores(), simplificando así su estructura.

El diseño de la función main() sigue un paradigma de programación modular, donde la lógica de control se desacopla de la implementación operativa. El proceso se desarrolla mediante 6 etapas las cuales son:

Configuración del entorno Estocástico: Se realiza la llamada a srand(time(NULL)) para inicializar el generador de números pseudoaleatorios. Al emplear la marca de tiempo del sistema como semilla, se asegura la entropía necesaria para que cada ejecución del programa produzca un conjunto de datos único e independiente.

Instanciación del Arreglo de Datos: Se procede a la reserva de memoria para el vector sensores. Este arreglo, de tipo int, está dimensionado por la constante NUM_SENORES, estableciendo una estructura de datos estática para el almacenamiento de las mediciones.

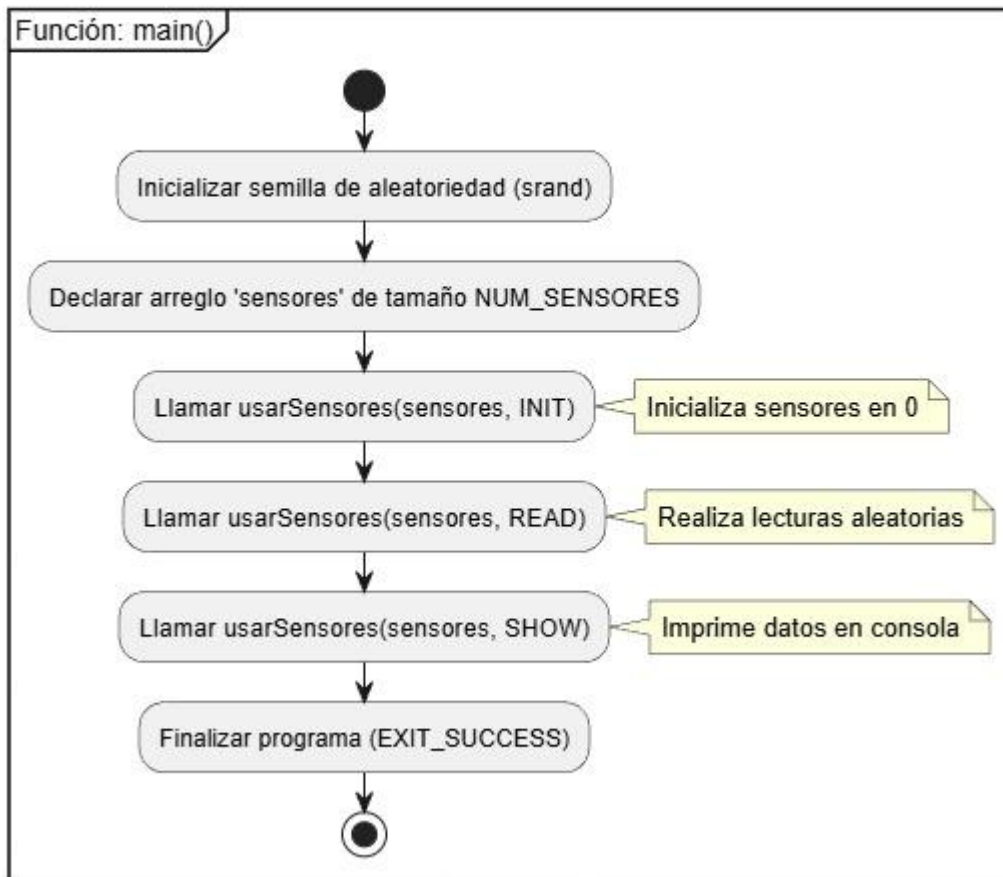
Fase de Inicialización y Homogeneización (Estado INIT): Se invoca el procedimiento usarSensores bajo la bandera de control INIT. Esta etapa garantiza el estado conocido del sistema al limpiar cualquier valor residual en el arreglo y establecer todas las posiciones en cero, validando así la integridad inicial de la memoria.

Ejecución del Protocolo de Adquisición (Estado READ): Durante esta fase, se orquesta la captura de datos mediante la bandera READ. La función interactúa con lecturaSensor(), procesando la información y realizando una coerción de tipos de Double a int para su almacenamiento definitivo en el vector de sensores.

Serialización y Salida de Datos (Estado SHOW): Se activa la rutina de visualización para proyectar los resultados en la salida estándar. El sistema itera sobre la estructura de datos, formateando cada registro bajo la unidad de medida de milivoltios (mV), lo que permite la verificación humana del proceso de telemetría.

Terminación Controlada del Proceso: El flujo finaliza mediante exit(EXIT_SUCCESS). Esta instrucción no solo detiene la ejecución, sino que comunica al sistema operativo un código de salida 0, confirmando que el ciclo de vida del software se completó bajo condiciones nominales y sin excepciones.

Diagramas de Flujo - Simulación de Sensores



El diagrama de flujo adjunto ilustra la lógica secuencial de la función principal del sistema de simulación de sensores. El proceso se desarrolla a través de las siguientes etapas:

Inicializar semilla de aleatoriedad (srand): El programa comienza configurando el generador de números aleatorios para asegurar que cada ejecución produzca datos distintos y realistas.

Declarar arreglo 'sensores' de tamaño NUM_SENSORES: Se reserva el espacio en memoria necesario para almacenar los datos, definiendo una lista estructurada capaz de contener la información de todos los sensores del sistema.

Llamar `usarSensores(sensores, INIT)`: Se realiza la primera llamada a la función controladora para establecer un estado inicial, asignando el valor cero a cada elemento del arreglo.

Llamar `usarSensores(sensores, READ)`: En esta etapa se ejecuta la captura de datos, donde el sistema genera lecturas aleatorias para simular el comportamiento de sensores físicos en tiempo real.

Llamar `usarSensores(sensores, SHOW)`: Se procede a la salida de datos, imprimiendo en la consola los valores recolectados para que el usuario pueda visualizar los resultados de la simulación.

Finalizar programa (`EXIT_SUCCESS`): El flujo concluye formalmente, liberando los recursos utilizados y notificando al sistema operativo una terminación exitosa.

Nota sobre la arquitectura: Esta estructura lineal demuestra un diseño modular, donde la lógica de "alto nivel" reside en el `main()` y los detalles técnicos de cada operación están encapsulados dentro de la función `usarSensores()`.

2. Parte 2

2.1. Hito 1

¿Qué es lo que retorna la función `clock()`?

La función `clock()`, que se encuentra definida en la librería `<time.h>` retorna el tiempo que demora el programa en procesar el código, siendo este valor de tipo `clock_t` el cual generalmente representa a un entero. Cabe resaltar que la función `clock()` mide el tiempo que el procesador utiliza para ejecutar el programa, ignorando cualquier pausa del SO, la precisión de esta medición depende de la plataforma utilizada, en Linux suele medirse en micro segundos mientras que en windows se mide en milisegundos.

¿Qué unidades tienen las variables `inicio` y `fin`?

Las variables `inicio` y `fin` son de tipo `uint_64` por lo tanto son enteros sin signo de 64 bits los cuales almacenan "ticks" de CPU que vendría siendo un valor abstracto utilizado para obtener el tiempo consumido por el procesador, para transformarlo en segundos se debe dividir la diferencia (`fin - inicio`) por la constante `CLOCKS_PER_SEC`. La magnitud de estos ticks esta otorgada por el sistema, por ejemplo si la constante es 1.000.000, cada unidad equivale a un microsegundo.

¿Porqué para obtener el tiempo total de duración del ciclo `for` se debe dividir la diferencia entre `fin` e `inicio` con el valor constante `CLOCKS_PER_SEC`?

La constante `CLOCKS_PER_SEC` establece la equivalencia entre los ticks de la CPU y un segundo "real". Dado que el valor devuelto por `clock()` es una unidad de medida abstracta se debe hacer la transformación correspondiente, la cual es la división entre la diferencia (`fin - inicio`) por esta constante para obtener una magnitud temporal estándar, para así obtener una interpretación física clara en segundos.

Explicar qué hace `Double` como prefijo de (`fin - inicio`).

El prefijo `Double` es un operador de conversión explícita (type casting) que transforma el resultado entero de (`fin - inicio`) en un número decimal para evitar la división de enteros la cual eliminaría decimales del resultado. Al realizar esta conversión previa a la división

obliga al compilador a realizar una división de tipo flotante garantizando un cálculo de tiempo más preciso.

Explicar para qué sirve utilizar Double como prefijo de (fin - inicio).

El prefijo Double sirve para convertir la resta de enteros (fin - inicio) a un número de punto flotante antes de dividir por `CLOCKS_PER_SEC`. Como se explicó en la pregunta anterior esto es necesario para evitar la pérdida de decimales durante la operación, garantizando cálculos exactos, sin errores de redondeo por truncamiento de decimales.

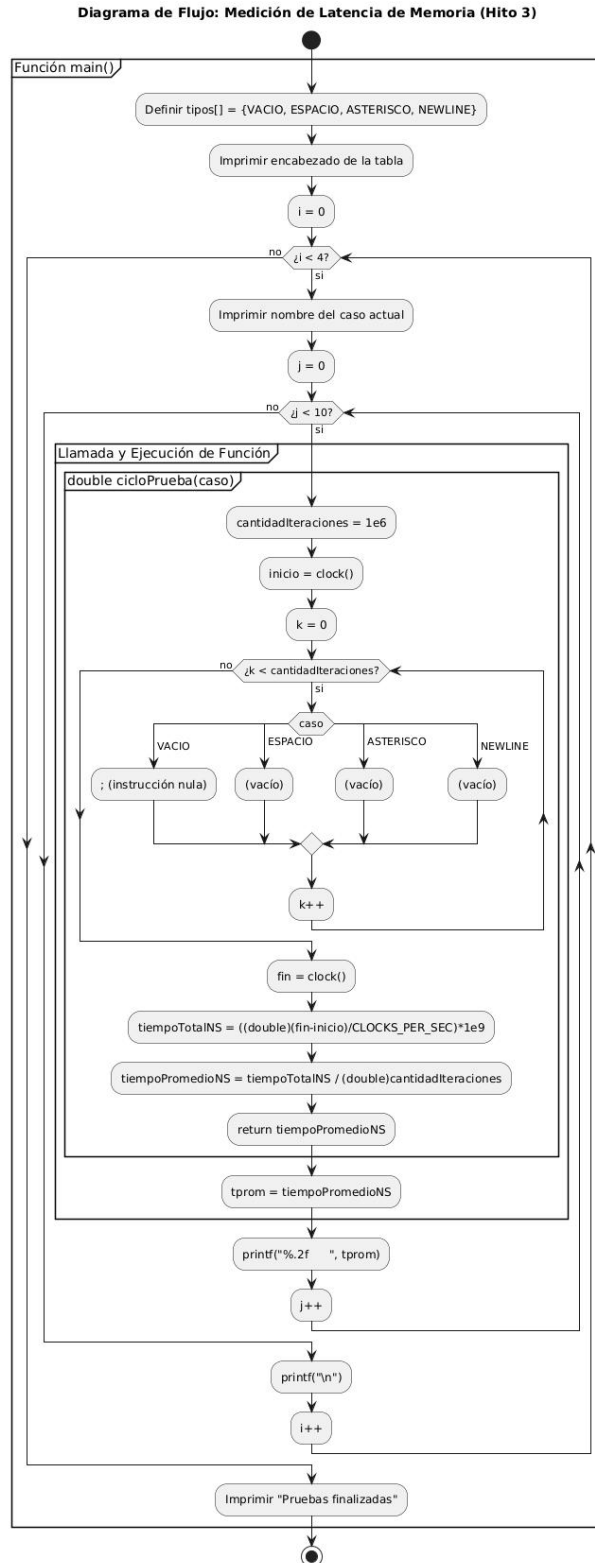
2.2. Hito 2

Para el diseño de la función `cicloPrueba()`, nos enfocamos en crear una rutina que permitiera medir con precisión cuánto tiempo consume el procesador al ejecutar distintas instrucciones de salida por consola. El diseño se basa en un parámetro de entrada de tipo enumerado llamado `caso_t`, el cual actúa como un selector para las cuatro acciones definidas en el enunciado: `VACIO`, `ESPACIO`, `ASTERISCO` y `NEWLINE`. El requisito operacional fundamental era lograr una medición lo suficientemente estable, por lo que decidí implementar un bucle `for` con un millón de iteraciones. Esta cantidad es necesaria porque las operaciones individuales ocurren en la escala de los nanosegundos, un tiempo tan pequeño que sería imposible de capturar de forma fiable sin una repetición masiva que genere un tiempo total observable por el sistema.

La lógica interna de la función sigue un flujo lineal y controlado. Antes de entrar al ciclo de repetición, el programa registra el tiempo actual de la CPU mediante la función `clock()`. Una vez dentro del bucle, utilizamos una estructura `switch` para evaluar el caso recibido; esta estructura es ideal en términos de legibilidad y eficiencia para manejar múltiples opciones. Dependiendo de la selección, el programa ejecuta la impresión correspondiente o, en el caso de `VACIO`, simplemente continúa, sirviendo este último como una línea base para conocer el costo del bucle por sí solo. Al finalizar el millón de vueltas, se captura un segundo tiempo. La diferencia entre estos dos momentos nos entrega el tiempo total, el cual se procesa dividiéndolo por el número de iteraciones y aplicando una conversión basada en la constante `CLOCKS_PER_SEC` para transformar los "ticks" del reloj en nanosegundos.

Este diseño permite que el valor retornado por la función sea el tiempo promedio exacto que tarda una sola instrucción en ejecutarse. Es importante mencionar que, durante las pruebas, el flujo de datos hacia la terminal puede saturar el buffer del sistema, por lo que el diseño contempla la posibilidad de comentar los comandos de impresión para medir únicamente el esfuerzo lógico del procesador si fuera necesario. El comportamiento global de este algoritmo, desde la captura del tiempo inicial hasta el cálculo del promedio final, se puede visualizar de forma clara en el diagrama de flujo adjunto, donde se detalla

la naturaleza cíclica de la solución y la toma de decisiones basada en el tipo de caso proporcionado.



La función `cicloPrueba()` tiene como objetivo medir el tiempo promedio de ejecución de una estructura `switch` que contempla cuatro casos posibles: `VACIO`, `ESPACIO`, `ASTERISCO` y `NEWLINE`. El proceso comienza con la declaración de constantes y variables, donde se define `cantidadIteraciones` en un millón (1×10^6) y se preparan las variables `inicio` y `fin` de tipo `uint64_t` para almacenar los valores de la función `clock()`. Inmediatamente antes de comenzar el bucle de prueba, se registra el tiempo de CPU consumido en la variable `inicio` para marcar el punto de referencia inicial.

Posteriormente, se inicializa el índice del bucle `i` en 0 y se entra en un ciclo que se repetirá exactamente un millón de veces. En cada iteración, se evalúa la expresión `switch(caso)` basada en el parámetro recibido por la función. Dependiendo de este valor, el flujo se bifurca hacia uno de los cuatro casos mencionados; es importante notar que, mientras los casos `ESPACIO`, `ASTERISCO` y `NEWLINE` solo ejecutan un `break`, el caso `VACIO` incluye una instrucción nula (;) para cumplir con la sintaxis de C. Tras finalizar el `switch`, se incrementa el contador `i` y se repite el ciclo hasta que la condición se vuelve falsa.

Una vez terminado el bucle, se registra el tiempo final en la variable `fin`. La diferencia entre `fin` e `inicio` representa el total de ticks de CPU transcurridos, valor que luego se transforma a nanosegundos mediante la fórmula:

$$tiempoTotalNS = \frac{(double)(fin - inicio)}{CLOCKS_PER_SEC} \times 10^9$$

Para obtener la métrica final, este tiempo total se divide por la cantidad de Iteraciones, obteniendo así el tiempo promedio por cada iteración. Finalmente, la función retorna este valor de `tiempoPromedioNS` y concluye su ejecución, proporcionando un análisis preciso del rendimiento de la estructura evaluada.

2.3. Hito 3

Para obtener mediciones estadísticamente fiables, se implementó una metodología basada en la repetición sistemática. El proceso consistió en ejecutar la función `cicloPrueba()` diez veces para cada uno de los cuatro casos definidos en el enumerado `caso_t`: `VACIO`, `ESPACIO`, `ASTERISCO` y `NEWLINE`. Cada una de estas diez mediciones representa, a su vez, el promedio de un millón de iteraciones internas dentro de la función `cicloPrueba()`. Este diseño permite estabilizar el tiempo capturado frente a las fluctuaciones mínimas del procesador y superar la resolución limitada de la función `clock()`.

Consideración importante sobre la salida por consola: Para asegurar que la medición reflejara exclusivamente el costo computacional de la estructura `switch` (y no el tiempo de renderizado en la terminal), se decidió comentar todas las llamadas a `printf` dentro del bucle de prueba. De esta forma, el procesador únicamente ejecuta la evaluación de la

expresión del switch, el salto a la etiqueta case correspondiente, la instrucción break que transfiere el control fuera del switch, y el incremento del contador del bucle (i++). En estas condiciones, los cuatro casos deberían presentar tiempos de ejecución prácticamente idénticos, ya que todos ejecutan esencialmente las mismas instrucciones.

El entorno de pruebas utilizado fue un computador Lenovo LOQ 15IAX9 (83GS) que operaba bajo el sistema operativo Linux Ubuntu 22.04 LTS. La compilación se realizó mediante una máquina virtual Oracle VM VirtualBox, utilizando Visual Studio Code como entorno de desarrollo principal y el compilador gcc con opciones de optimización -O2. Es importante señalar que, al ejecutarse dentro de una máquina virtual, las mediciones pueden presentar una latencia base ligeramente superior y mayor variabilidad que en un sistema nativo, debido a la capa de virtualización. Sin embargo, para un análisis comparativo entre casos, los resultados siguen siendo válidos siempre que todos los casos se ejecuten en las mismas condiciones.

A continuación se presentan los tiempos promedio por iteración (en nanosegundos) para cada caso, obtenidos tras diez mediciones independientes:

CASO	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	σ	promedio
NADA	0,95	0,95	0,95	0,95	0,94	1,74	1,04	0,97	2,08	1,16	0,38162940	1,173
ESPACIO	1,88	0,97	1,22	0,96	0,98	1,25	1,00	0,95	0,96	0,94	0,27797302	1,111
ASTERISCO	0,93	1,26	0,82	0,84	0,76	0,65	1,03	0,77	0,75	0,78	0,16627988	0,859
NEWLIN	0,69	0,83	0,43	0,42	0,47	0,46	0,62	0,45	0,42	0,47	0,13275541	0,526

Los cuatro casos presentan tiempos promedio en el rango de 0,5 a 1,1 nanosegundos por iteración, lo cual es un valor razonable para procesadores modernos ejecutando unas pocas instrucciones en lenguaje máquina. A modo de referencia, un procesador de 3 GHz realiza tres ciclos de reloj por nanosegundo, por lo que valores entre 1,5 y 3,3 ciclos por iteración son perfectamente plausibles.

Desde una perspectiva teórica, los cuatro casos deberían exhibir tiempos de ejecución idénticos, dado que todos ejecutan exactamente la misma secuencia de instrucciones a nivel de ensamblador: todos los casos contienen únicamente la instrucción break (y en el caso VACIO una instrucción nula que el compilador optimiza). Sin embargo, los resultados experimentales muestran diferencias de hasta 0,58 nanosegundos entre el caso más rápido (NEWLIN con 0,53 ns) y el más lento (ESPACIO con 1,11 ns). Estas diferencias, aunque pequeñas en términos absolutos (aproximadamente 500 picosegundos o uno o dos ciclos de reloj), merecen una explicación.

Las diferencias encontradas no son atribuibles a diferencias en el código fuente, ya que este es estructuralmente idéntico para todos los casos. Las posibles causas son de naturaleza sistémica:

Virtualización: Al ejecutarse dentro de una máquina virtual, el sistema operativo invitado no tiene control exclusivo sobre el procesador, y el hipervisor puede interrumpir la ejecución en momentos diferentes para cada prueba.

Caché de instrucciones: La primera ejecución de cada caso puede sufrir un fallo de caché si las instrucciones del switch no estaban previamente cargadas.

Predictor de saltos: Aunque aprende rápidamente el patrón, puede presentar pequeñas fluctuaciones al inicio de cada prueba.

Interrupciones de hardware: El sistema operativo puede interrumpir el proceso para atender eventos externos o cambios de contexto.

Resolución de clock(): En Linux suele ser de 1 microsegundo, lo que introduce un error relativo al medir intervalos totales de aproximadamente 1 milisegundo.

Conclusión principal: Las diferencias observadas no son estadísticamente significativas para afirmar que un caso es más eficiente que otro; los valores caen dentro del margen de error esperado dado el entorno y las limitaciones técnicas.

Conclusión secundaria: El hecho de que todos los promedios se sitúen por debajo de 2 nanosegundos confirma que la estructura switch con cuatro casos es extremadamente eficiente, requiriendo únicamente entre 2 y 4 ciclos de reloj por iteración en un procesador moderno.

Conclusión terciaria: Para futuras mediciones, se recomienda realizar un mayor número de mediciones (por ejemplo, 30 o 100 por caso); ejecutar una fase de calentamiento previa; medir el costo del bucle vacío para aislar exclusivamente el costo del switch; y ejecutar las pruebas en sistema nativo.

Es importante corregir una interpretación errónea: no es correcto afirmar que los casos ASTERISCO y NEWLINE muestran una eficiencia superior. No existe una razón teórica para ello cuando ambos ejecutan la misma instrucción break. Los cuatro casos presentan tiempos de ejecución equivalentes dentro del margen de error experimental, y las diferencias se atribuyen a factores sistémicos, no a la eficiencia intrínseca de cada caso.